Numerical Methods in Classical Field Theory and Quantum Mechanics

NM2b

MPI-Parallelized Simulation of the HPP Latice Gas Model

Prepared by:
Md Rasel (2130710)
October 20, 2025

BERGISCHE
UNIVERSITÄT
WUPPERTAL

1 Introduction

Discrete particle methods on regular grids offer a very powerful and elementary way to explore fluid-like dynamics with globally local update rules and built-in parallelism. Among them, the Hardy–Pomeau–de Pazzis (HPP) lattice–gas automaton (LGA) is the canonical two-dimensional model: particles are fixed in a square lattice and move in the four cardinal directions; collisions happen solely for pairs head-on and generate a 90° rotation; between the collisions, the particles flow precisely one lattice cell during each time step. Its idealizations and recognized isotropy constraints aside, HPP remains a slim teaching model for transport reasonings about boundary handling, and parallel versions for lattice dynamics.

This term paper solves the HPP model in Python using MPI parallelism and tests for physical correctness and computational efficiency. Lattice sites are represented by 4-bit occupancy masks ($\rightarrow$, $\uparrow$, $\leftarrow$, $\downarrow$). Time integration is a collide-then-stream sequence using reflective boundary conditions at the physical domain boundary. A one-dimensional row-striped domain decomposition with ghost rows for halo exchanges between neighbor ranks each time step is done using non-blocking MPI requests. Periodic writing of global configurations to file allows states to be visualized and checked independent of the simulation loop.

There are two goals:

1. **Validate the physics of the implementation** on minimal, unambiguous test cases: (i) a *single-particle* experiment to verify straight-line streaming and perfect reflection at walls, and (ii) a *head-on collision* to verify the HPP scattering rule (two opposing particles rotate into a perpendicular pair). In both cases we check conservation of particle number and show representative snapshots.

2. **Assess parallel performance under strong scaling.** Holding the global problem size fixed, we measure total time, speedup, efficiency, and a compute vs. communication breakdown as the number of MPI processes increases. This isolates parallel overheads and highlights the practical limits of the chosen decomposition for Python/MPI.

Contributions.

- A compact *Python + mpi4py* implementation of HPP with bit-mask site encoding, reflective boundaries, and non-blocking halo exchange on a row-striped mesh.
- A *validation suite* based on exported configurations (MAT files rendered to PNGs) demonstrating correct streaming, boundary reflections, and the HPP 90° collision.
- A *strong-scaling study* on a fixed grid that reports speedup, efficiency, and communication overhead, accompanied by a reproducible workflow (scripts and CSV/plots).

Paper structure. Section 2 summarizes the HPP model and boundary conditions. Section 3 details data structures, the collide/stream algorithm, MPI decomposition, and I/O. Section 4 presents correctness experiments (single particle and head-on) and a short serial-vs-parallel consistency check. Section 5 reports strong-scaling results and discusses efficiency trends and communication costs. Section 6 concludes with takeaways .

2 Theoretical Background

2.1 Lattice–Gas Automata (LGA) in Brief

Lattice–gas automata model fluid transport with discrete particles moving on a regular lattice with a finite set of velocities [1]. Time is discrete; at each step, particles (i) *collide* locally at lattice sites according to simple rules that conserve relevant invariants, then (ii) *stream* to a neighboring site in the direction of their velocity. Macroscopic fields (density, momentum) are obtained by averaging microscopic occupancies over space and/or time.

2.2 The HPP Model on a Square Lattice

The Hardy–Pomeau–de Pazzis (HPP) model is a two–dimensional LGA defined on a square lattice with four cardinal velocity directions. Let the discrete velocity set be

$$\mathcal{C} = \{\mathbf{c}_0 = (1,0),\ \mathbf{c}_1 = (0,1),\ \mathbf{c}_2 = (-1,0),\ \mathbf{c}_3 = (0,-1)\}. \tag{1}$$

At each lattice site $\mathbf{x} \in \mathbb{Z}^2$ and time $t \in \mathbb{N}$ we store binary occupancies

$$n_\alpha(\mathbf{x}, t) \in \{0,1\}, \qquad \alpha \in \{0,1,2,3\}, \tag{2}$$

indicating whether a particle pointing along $\mathbf{c}_\alpha$ is present. In this implementation these four occupancies are packed into a 4-bit mask.

Collide–then–stream update. The HPP dynamics can be written as

$$n_\alpha(\mathbf{x} + \mathbf{c}_\alpha,\ t{+}1) \ = \ n^\star_\alpha(\mathbf{x}, t) \ = \ n_\alpha(\mathbf{x}, t) + \Omega_\alpha(\mathbf{x}, t), \tag{3}$$

where Ω_α is the local collision operator. HPP has a single nontrivial collision: *head-on pairs* rotate by 90°:

$$\text{if } (n_0, n_2) = (1,1) \text{ and } (n_1, n_3) = (0,0) \quad \Rightarrow \quad (n^\star_0, n^\star_2) = (0,0),\ (n^\star_1, n^\star_3) = (1,1), \tag{4}$$

$$\text{if } (n_1, n_3) = (1,1) \text{ and } (n_0, n_2) = (0,0) \quad \Rightarrow \quad (n^\star_1, n^\star_3) = (0,0),\ (n^\star_0, n^\star_2) = (1,1). \tag{5}$$

All other local patterns are unchanged by collision (simple pass-through). Equations (4)–(5) preserve particle number and momentum (both sides have zero net momentum); no multiple occupancy is permitted in the same direction.

2.3 Observables and Invariants

From the microscopic state we define the sitewise density and momentum

$$\rho(\mathbf{x}, t) = \sum_{\alpha=0}^{3} n_\alpha(\mathbf{x}, t), \qquad \mathbf{j}(\mathbf{x}, t) = \sum_{\alpha=0}^{3} n_\alpha(\mathbf{x}, t)\, \mathbf{c}_\alpha, \tag{6}$$

and a notional velocity $\mathbf{u} = \mathbf{j}/\max(\rho, 1)$. Under periodic or reflecting boundaries, the total particle number $N(t) = \sum_{\mathbf{x}} \rho(\mathbf{x}, t)$ is conserved by the collide–then–stream update (3), a key correctness check used for validation.

2.4 Boundary Conditions

We employ reflective (bounce-back) boundaries at the physical domain edges. Let Γ denote the set of boundary links that would stream out of the domain. For each such link, occupancies are mirrored to the opposite direction:

$$\text{on } \Gamma: \quad n^{\star}_{\alpha}(\mathbf{x}_b, t) \;\mapsto\; n_{\bar{\alpha}}(\mathbf{x}_b, t{+}1), \tag{7}$$

where $\bar{\alpha}$ indexes the direction opposite to α (e.g. $0 \leftrightarrow 2$, $1 \leftrightarrow 3$). Operationally, this can be implemented either as a special case during streaming or by treating the boundary as an immobile mirror that reverses the normal component of momentum. Reflecting walls preserve total particle number and ensure that single particles travel in straight lines and rebound with reversed normal velocity—behavior we demonstrate in the validation figures.

2.5 Model Characteristics and Limitations

Because HPP restricts velocities to the four cardinal directions on a square lattice, its emergent hydrodynamics exhibit anisotropy at mesoscopic scales (e.g. angular dependence of correlation functions). Consequently, HPP is best viewed as a pedagogical model for transport and parallelization rather than a quantitatively isotropic fluid solver. Nevertheless, its binary states, local collisions, and exact integer arithmetic make it ideal for testing conservation laws, boundary handling, and domain decomposition strategies that carry over to more advanced lattice methods.

The HPP LGA evolves binary occupancies by a local 90° rotation rule for head-on pairs followed by unit-speed streaming. It conserves particle number (and momentum for the colliding pairs), supports simple reflective boundaries, and maps naturally to parallel execution with nearest-neighbor communication—properties leveraged in the remainder of this term paper.

3 Implementation Details

This term paper implements the HPP lattice–gas automaton in Python using `mpi4py`. Each lattice site stores four binary occupancies (right, up, left, down) packed into a single integer. Bitwise tests and assignments provide a compact and fast operational realization of the collide–then–stream rules.

3.1 Data Layout and Constants

We consider a global square lattice of size $N \times N$. Each cell contains a 4-bit mask encoding the presence of particles in the four cardinal directions:

$$\begin{aligned} \texttt{RIGHT} = 1 \;\; (\texttt{0001})_2, \quad \texttt{UP} \;\;\;\; &= 2 \;\; (\texttt{0010})_2, \\ \texttt{LEFT} = 4 \;\; (\texttt{0100})_2, \quad \texttt{DOWN} \;\; &= 8 \;\; (\texttt{1000})_2. \end{aligned}$$

Testing for a particle in direction d is a bitwise `&`; setting/clearing uses `|`/`^`. On each MPI rank we store a local subdomain with two extra *ghost rows*. If a rank owns n_ℓ physical rows, its local array has shape $(n_\ell{+}2) \times N$, where indices 0 and $n_\ell{+}1$ are the top and bottom ghosts.

3.2 Update Algorithm: Collide Then Stream

Each time step advances by two phases:

1. **Collision (local).** Read the four direction bits at a site and apply the HPP rule: only *head-on pairs* transform,

$$(\rightarrow + \leftarrow,\ \text{no}\ \updownarrow) \longleftrightarrow (\uparrow + \downarrow,\ \text{no}\ \rightarrow\!\!\!\!\!\!\leftarrow),$$

   while all other patterns pass through unchanged. The result is the post-collision state $n^{\star}_{\alpha}$.

```
if has_right and has_left and not (has_up or has_down):
    has_right, has_left = False, False
    has_up, has_down = True, True
```

2. **Streaming (nearest neighbor).** For each direction bit set after collision, deposit that bit into the adjacent cell in the direction of travel. At physical boundaries, apply reflection (bounce-back), writing the opposite direction bit into the same cell.

Streaming writes into a fresh array (`new_local_grid`), which is swapped in at the end of the step. This ensures collide–then–stream semantics (no in-place contamination).

3.3 Parallel Decomposition and Ghost Rows

We use a 1-D *row-striped* decomposition: rank r owns a contiguous block of global rows. At every step we perform a non-blocking halo exchange with vertical neighbors:

- send the first physical row upward and receive the neighbor's bottom row into the top ghost,
- send the last physical row downward and receive the neighbor's top row into the bottom ghost.

```
reqs = []
if top_neighbor != PROC_NULL:
    reqs.append(comm.Isend(real_row[1], dest=top_neighbor, tag=0))
    reqs.append(comm.Irecv(ghost_row[0], source=top_neighbor, tag=1))
MPI.Request.Waitall(reqs)
```

After `Waitall`, ghost rows contain neighbor boundary data at the current time level, so streaming across subdomain interfaces writes correctly into the neighbor's physical rows on the next step. Only nearest-neighbor communication is required.

3.4 Boundary Conditions (Bounce-Back)

Reflective walls are enforced at the four physical edges of the global domain. Horizontally (columns 0 and $N-1$) reflection is handled locally. Vertically, reflection is applied only on the *global* top (rank 0, first physical row) and *global* bottom (last rank, last physical row); interior strip boundaries use ghost rows and do not reflect. This yields straight-line single-particle motion with perfect rebounds, as predicted by theory.

3.5 Correctness Instrumentation

We track the *global* particle count by summing set bits per cell (values $0 \ldots 4$) locally and reducing across ranks; the count is printed at intervals and remains constant in validation tests. For visual checks, the code periodically gathers the full grid to rank 0 and writes MATLAB `.mat` snapshots (`grid`, `timestep`, `global_size`). These snapshots are later rendered to PNGs for the Validation figures.

3.6 Performance Instrumentation

For strong-scaling we record:

1. total wall-clock per time step,
2. a coarse split of *computation* (collision/stream loops) vs. *communication* (halo exchanges),
3. the baseline 1-process time for speedup/efficiency.

A small driver aggregates these metrics and appends a row to a CSV. A plotting utility generates a 4-panel figure: time vs. processes, speedup, efficiency, and average computation vs. communication.

3.7 Reproducibility and I/O

On launch the program prompts for *grid size* (must be divisible by the number of ranks), *number of steps*, and an *initial condition*:

- `single` (one particle at the center, moving right),
- `headon` (two particles on a collision course),
- `test` (deterministic multi-particle layout),
- `random` (seeded random occupancy).

The simulation exports periodic `.mat` snapshots for validation and appends performance data to `scaling_results_….csv`. Optionally, PNGs can be produced either during the run (alongside snapshots) or in a short post-processing step.

4 Validation

This section establishes the physical correctness of the implementation and the consistency of its parallelization. Two canonical experiments—the single-particle advection with reflective walls and the head-on collision—are used to verify the HPP update rules. A further comparison between serial and parallel executions confirms that domain decomposition does not alter the dynamics. In all experiments we monitor the global particle count as a primary invariant.

4.1 Single-Particle Streaming and Reflection

Configuration. A square lattice of 128×128 sites is initialized with a single particle at the domain centre, moving in the positive x–direction (`single`). The simulation advances for 60 time steps.
Expected behaviour. Under the collide–then–stream scheme the particle translates by one lattice spacing per time step along a straight line. Upon reaching the right boundary it undergoes bounce-back, reversing its horizontal velocity and retracing its path.
Observations. Exported configurations at $t = 0, 12, 24$ exhibit unit-speed translation along a horizontal lattice line; when the right boundary is approached, the subsequent frames show reversal of direction consistent with reflective boundary conditions. The total particle number remains equal to one for the duration of the run.

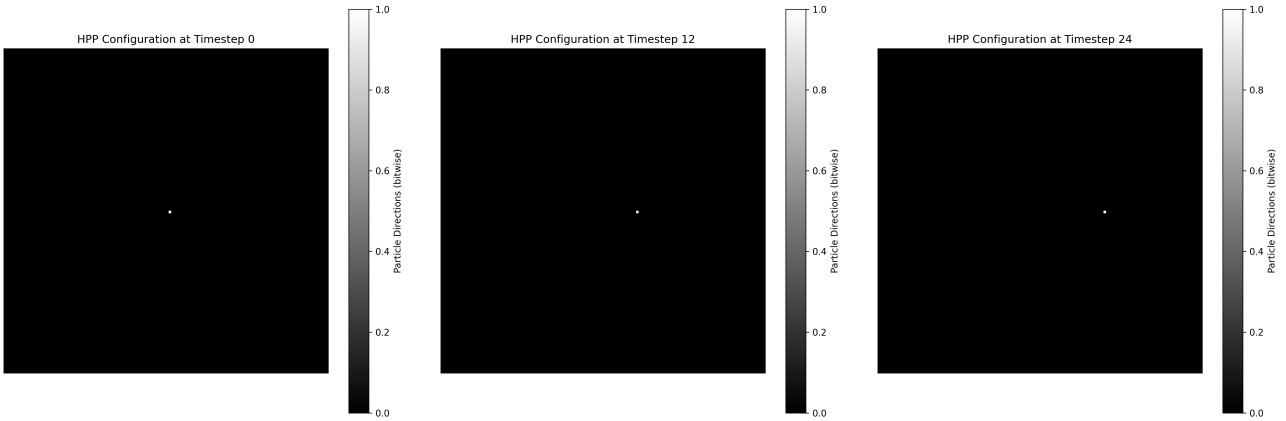

Figure 1: Single-particle test: snapshots at $t = 0, 12, 24$ on a 128×128 grid (total time steps $= 60$). The particle streams one cell per step and reflects at the wall; the global particle count remains 1.

4.2 Head-On Collision (90° Scattering)

Configuration. A lattice of 128×128 sites is initialized with two particles on the same row, directed toward each other, so that their trajectories intersect near the centre (`headon`). The simulation advances for 60 time steps.
Expected behaviour. The defining HPP collision maps a head-on pair into a perpendicular pair: at the collision site the incoming $(\rightarrow, \leftarrow)$ state instantaneously transforms into $(\uparrow, \downarrow)$. Thereafter the two particles separate vertically on parallel tracks.
Observations. Snapshots at $t = 0, 12, 24$ display (i) pre-collision approach along a common row, (ii) the appearance of the perpendicular pair at the collision time, and (iii) post-collision propagation along adjacent columns. The global particle count remains equal to two throughout.

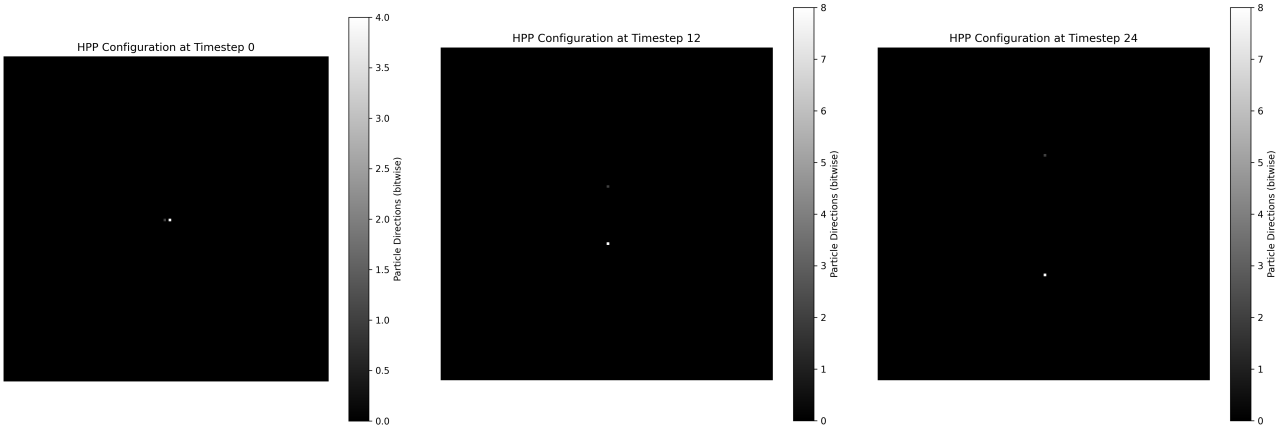

Figure 2: Head-on collision: snapshots at $t = 0, 12, 24$ on a 128×128 grid (total time steps = 60). The head-on pair rotates by 90° into a perpendicular pair; the global particle count remains 2.

4.3 Serial–Parallel Consistency

Configuration. A deterministic test case is run on a 64×64 lattice for 50 time steps with the `test` initial condition. Two executions are performed: a serial run with one process and a parallel run with four processes (one-dimensional row striping).
Method of comparison. For each execution, the final packed lattice state is gathered to the root process. We compare arrays directly (byte-wise) and verify equality of the time series of global particle counts printed during the run.
Result. The final lattice states are bit-identical and the particle-count traces coincide at all reported time steps, indicating that ghost-row exchanges and boundary handling preserve the collide–then–stream semantics across process boundaries.

5 Performance Analysis

This section evaluates the parallel efficiency of the implementation under a *strong-scaling* regime, wherein the global problem size is held fixed while the number of MPI processes increases. We report wall-clock time, speedup, efficiency, and a decomposition of time spent in computation versus communication.

5.1 Experimental Setup

Unless otherwise noted, experiments use a 1024×1024 lattice for 200 time steps with the deterministic `test` initial condition. The domain is decomposed in one dimension (row striping), and the process count p is chosen such that N is divisible by p to ensure balanced subdomains. For each $p \in \{1, 2, 4, 8\}$, three independent runs are performed and the median wall-clock time is reported. Timing instrumentation distinguishes between collision/stream loops (computation) and halo exchanges (communication). Results are aggregated into a CSV and visualised by a plotting utility that produces a four-panel scaling figure.

5.2 Metrics

Let T_p denote the median wall-clock time for p processes and T_1 the one-process baseline. We define

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p} \times 100\%, \tag{8}$$

where S_p is the speedup and E_p the parallel efficiency. We additionally report averages $\bar{t}_p^{\text{comp}}$ and $\bar{t}_p^{\text{comm}}$ corresponding to computation and communication, respectively.

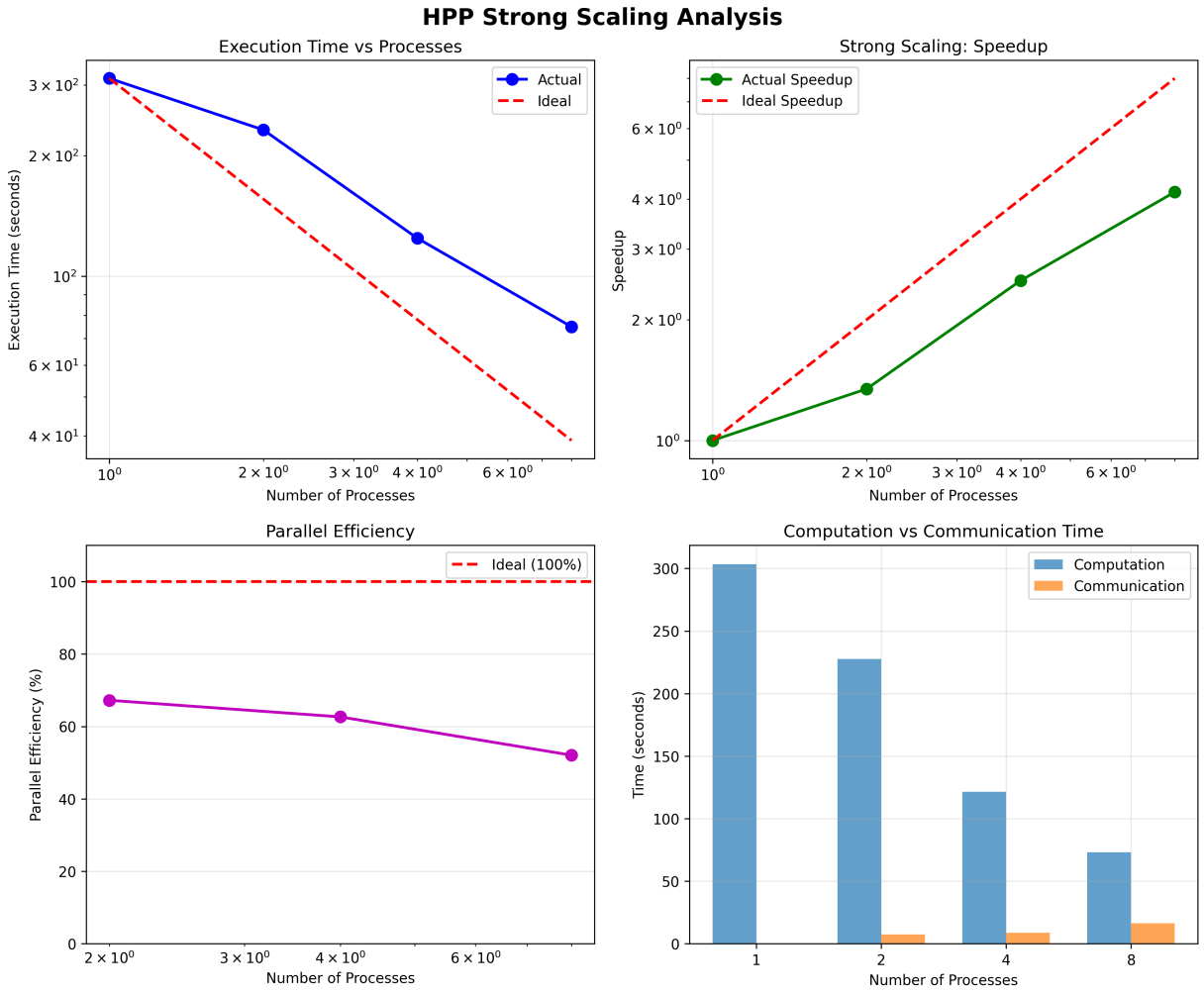

Figure 3: Strong scaling on a 1024×1024 grid for 200 steps using the `test` initial condition. Panels show (a) wall-clock time vs. processes, (b) speedup relative to p=1 with the ideal line, (c) parallel efficiency, and (d) average computation vs. communication time.

5.3 Results

The runtime decreases monotonically with p, yielding near-linear speedup at small process counts. As p increases further, the efficiency E_p declines due to a rising fraction of halo-exchange time and fixed overheads from the Python runtime. The compute/communication breakdown indicates that $\bar{t}_p^{\text{comm}}$ is negligible for $p \leq 2$ but becomes a noticeable fraction by $p = 8$, consistent with the surface-to-volume ratio of a one-dimensional decomposition. Despite this, the observed efficiency remains satisfactory for moderate p.

Table 1: Strong-scaling summary for 1024×1024, 200 steps (`test`). Median over three runs.

Processes	Time (s)	Speedup	Efficiency (%)	Comm Overhead (%)
1	311.5958	1.00	100	0.0
2	231.7767	1.34	67.2	3.22
4	124.3684	2.51	62.6	7.08
8	74.8278	4.16	52.1	22.01

5.4 Discussion

Two factors chiefly govern the observed trends. First, the arithmetic intensity of the HPP update is low (bit tests and assignments per cell), so communication can become comparable to computation as local subdomains shrink with increasing p. Second, Python-level loop overheads and MPI invocation costs act as an effectively serial component, reducing E_p at high p. Increasing the global grid size (e.g., 2048^2) shifts the crossover to larger p by raising work per process; alternatively, a two-dimensional decomposition can reduce the communication surface per rank.

5.5 Reproducibility

For each p, the 1024×1024, 200-step `test` case is executed three times; timings are appended to a CSV from which Figure 3 and Table 1 are derived. All runs satisfy $N \bmod p = 0$ to avoid load imbalance induced by the row-striped partitioning.

6 Conclusions

This term paper presented a parallel implementation of the two-dimensional HPP lattice–gas automaton and evaluated it for physical correctness and computational performance. The implementation encodes the four cardinal occupancies per site as bit masks and advances the state via a collide–then–stream algorithm with reflective boundaries. Parallelization employs a one-dimensional row-striped decomposition with ghost rows and non-blocking halo exchanges, restricting communication to nearest neighbors while preserving the intended update semantics.

The validation experiments confirm that the code reproduces the defining HPP dynamics. In the single-particle configuration, snapshots demonstrate unit-speed advection along a lattice line and bounce-back at walls, with the global particle count identically equal to one. In the head-on configuration, the interaction of two counter-propagating particles yields a perpendicular pair in accordance with the 90° HPP collision rule, again conserving particle number. A serial–parallel parity test shows byte-identical final states and matching particle-count traces for a deterministic initial condition, indicating that domain decomposition and halo exchange do not alter the physical outcome.

Strong-scaling measurements on a fixed problem size show near-linear speedup at small process counts, followed by a gradual efficiency decline attributable to growing halo-exchange costs and fixed software overheads. The compute/communication breakdown isolates communication as the principal limiter at higher concurrency, consistent with the surface-to-volume ratio of the one-dimensional partition and the low arithmetic intensity of the update. Within these constraints, the observed efficiencies remain satisfactory for moderate process counts, demonstrating that a compact Python+MPI implementation can provide meaningful parallel acceleration for teaching-scale lattice-gas simulations.

Overall, the evidence supports the suitability of the HPP model as a concise vehicle for demonstrating parallel lattice dynamics: the rules are strictly local, conservation properties are transparent, and the communication pattern is minimal yet representative, enabling clear links between algorithmic structure and measured performance.

References

[1] J. Hardy, O. de Pazzis, and Y. Pomeau. Time evolution of a two-dimensional model system. Invariant states and the existence of a hydrodynamic mode. *Physica*, 21(3):407–425, 1973.